Sesión 15: Introducción a la Gestión de Estado (Concepto de Estado Efímero vs. Global, Limitaciones de setState y Prop Drilling)

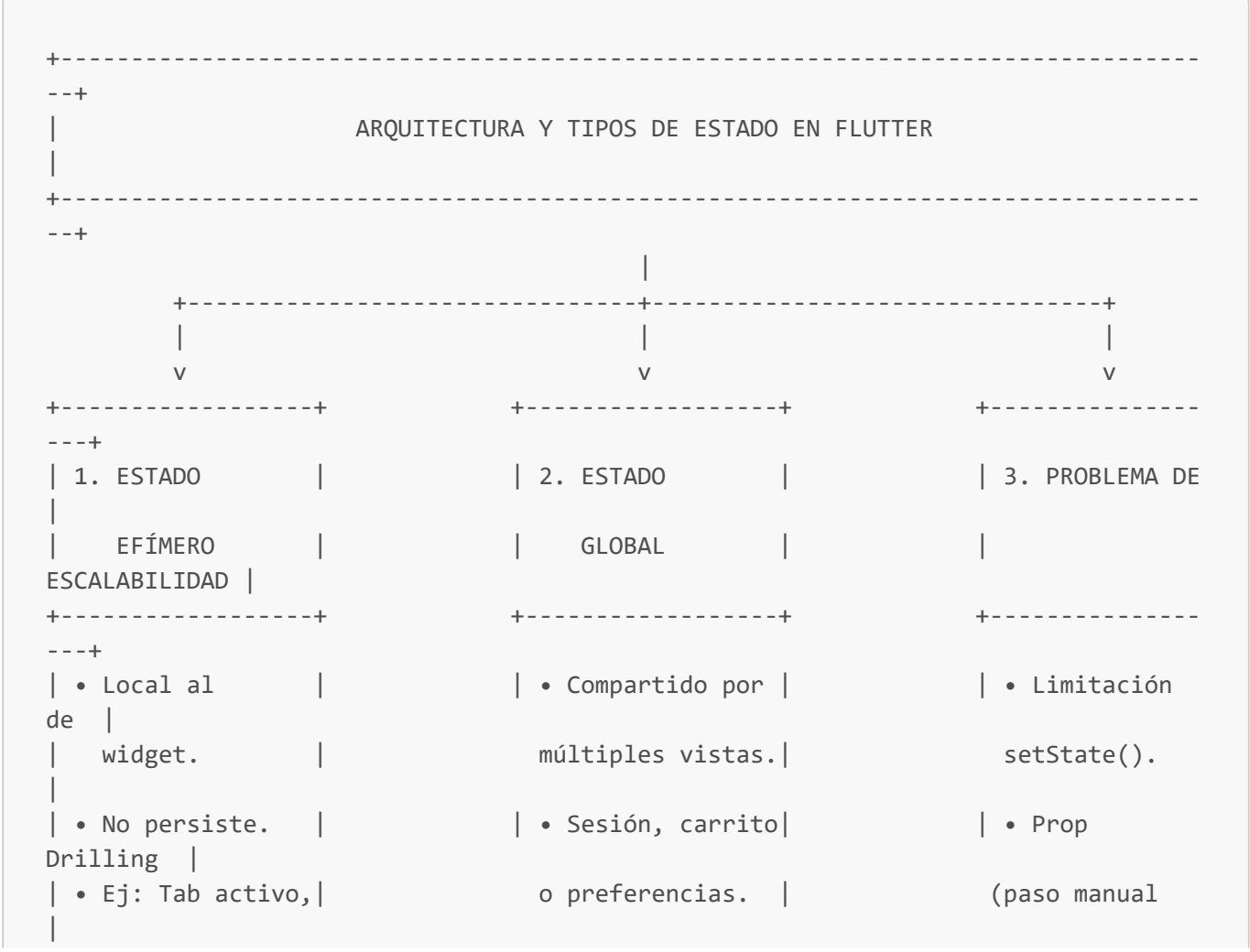
Introducción

La gestión de estado es el núcleo sobre el cual se construyen las aplicaciones reactivas en Flutter. Un estado representa la información requerida para reconstruir la interfaz de usuario en cualquier punto en el tiempo. A medida que una aplicación crece en complejidad, determinar dónde vive la información y cómo se propaga a través del árbol de widgets se convierte en un desafío arquitectónico crítico.

En esta sesión iniciaremos el Bloque III enfocado en la gestión de estado avanzada. Analizaremos las diferencias fundamentales entre **estado efímero** y **estado global**, exploraremos las limitaciones de `setState()` al escalar aplicaciones y examinaremos el problema del *Prop Drilling*, sentado las bases conceptuales para justificar el uso de gestores de estado estructurados.

Conceptos y Definiciones

El diseño de la interfaz en Flutter se rige por la ecuación declarativa $UI = f(state)$, donde la interfaz es una función directa del estado actual.



```
   textInput.     |             | • Requiere       |              de callbacks).
|
| • setState().    |               patrones.        |             | • Re-renders
|
|                  |             |                  |              innecesarios.
|
+------------------+             +------------------+             +---------------
---+
```

1. Estado Efímero (*Ephemeral State*) vs. Estado Global (*App State*)

- **Estado Efímero (Local):**
- Es la información contenida dentro de un único widget y que rara vez necesita ser accedida por otros componentes del árbol.
- No requiere arquitectura compleja ni persiste al destruir el widget.
- *Ejemplos:* El índice seleccionado en un `BottomNavigationBar`, la animación en curso de un `AnimatedContainer`, o el texto temporal ingresado en un `TextField`.
- *Manejo adecuado:* Un `StatefulWidget` nativo administrado mediante `setState()`.
- **Estado Global / de Aplicación (*App State*):**
- Información que debe ser compartida a través de múltiples pantallas o secciones independientes del árbol de widgets.
- *Ejemplos:* Token de autenticación del usuario, perfil de sesión, carrito de compras, o preferencias del tema de la aplicación.
- *Manejo adecuado:* Patrones de gestión de estado estructurados (`InheritedWidget`, Provider, Riverpod, BLoC).

---

2. Limitaciones de `setState()` al Escalar

El método `setState()` es la herramienta nativa para notificar al framework que el estado interno de un `StatefulWidget` ha cambiado. Sin embargo, presenta limitaciones severas en aplicaciones complejas:

- **Acoplamiento de Lógica y Presentación:** Fuerza a colocar reglas de negocio dentro de las clases de la interfaz de usuario (`State`), violando el principio de responsabilidad única.
- **Reconstrucciones Innecesarias:** Invocaciones a `setState()` marcan todo el subárbol del widget como "sucio" (*dirty*), provocando la ejecución del método `build()` en componentes hijos que no sufrieron cambios.
- **Falta de Persistencia:** Al desapilar o destruir el `StatefulWidget`, el estado local se destruye por completo.

---

3. El Problema del *Prop Drilling*

El *Prop Drilling* ocurre cuando se debe pasar información o funciones de respuesta (*callbacks*) a través de múltiples niveles de widgets intermedios que no necesitan esa información, únicamente para hacerla llegar a un widget descendiente profundo.

- **Efecto en el Código:** Genera constructores altamente verbosos, acoplamiento rígido entre componentes e incrementa exponencialmente la probabilidad de errores en el mantenimiento del código.

Ejemplo Completo con Código Fuente: Demostración del Anti-Patrón Prop Drilling y Paso Manual de Estado

El siguiente código implementa una aplicación que simula el flujo de un carrito de compras pasando el estado e incrementando contadores manualmente a través de cuatro niveles de profundidad de widgets. Esto permite evidenciar de manera clara la complejidad y fricción generadas por el *Prop Drilling*.

Código Fuente Explicado (`lib/main.dart`)

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const DemoPropDrillingApp());
}

/// Modelo de datos simple
class Producto {
  final String id;
  final String nombre;
  final double precio;

  const Producto({
    required this.id,
    required this.nombre,
    required this.precio,
  });
}

/// WIDGET NIVEL 1: Raíz de la Aplicación (Mantiene el Estado Global a la fuerza)
class DemoPropDrillingApp extends StatefulWidget {
  const DemoPropDrillingApp({super.key});

  @override
  State<DemoPropDrillingApp> createState() => _DemoPropDrillingAppState();
}

class _DemoPropDrillingAppState extends State<DemoPropDrillingApp> {
  // Estado que debería ser Global, forzado a vivir en la Raíz
  int _contadorCarrito = 0;

  void _agregarAlCarrito() {
    setState(() {
```

```
      _contadorCarrito++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Introducción a Gestión de Estado',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: PantallaCatalogoNivel1(
        totalCarrito: _contadorCarrito,
        onAgregarProducto: _agregarAlCarrito,
      ),
    );
  }
}

/// WIDGET NIVEL 2: Pantalla Catálogo (Intermedio - Sufre Prop Drilling)
class PantallaCatalogoNivel1 extends StatelessWidget {
  final int totalCarrito;
  final VoidCallback onAgregarProducto;

  const PantallaCatalogoNivel1({
    super.key,
    required this.totalCarrito,
    required this.onAgregarProducto,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Catálogo ESCOM'),
        backgroundColor: Theme.of(context).colorScheme.primary,
        foregroundColor: Colors.white,
        actions: [
          // Muestra el contador transmitido
          Badge(
            label: Text('$totalCarrito'),
            child: const Icon(Icons.shopping_cart),
          ),
          const SizedBox(width: 24),
        ],
      ),
      body: SeccionProductosNivel2(
        // Prop Drilling: Pasa propiedades hacia el nivel 3
        onAgregarProducto: onAgregarProducto,
      ),
```

```
    );
  }
}

/// WIDGET NIVEL 3: Sección de Productos (Intermedio - No usa los datos, solo los
pasa)
class SeccionProductosNivel2 extends StatelessWidget {
  final VoidCallback onAgregarProducto;

  const SeccionProductosNivel2({
    super.key,
    required this.onAgregarProducto,
  });

  @override
  Widget build(BuildContext context) {
    const productoEjemplo = Producto(
      id: 'DART-BOOK',
      nombre: 'Libro de Texto: Dart 3 & Flutter',
      precio: 450.00,
    );

    return Padding(
      padding: const EdgeInsets.all(20.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          const Text(
            'Demostración de Prop Drilling',
            style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
          ),
          const SizedBox(height: 8),
          const Text(
            'Este widget "SeccionProductosNivel2" NO requiere la función
"onAgregarProducto", '
            'pero está obligado a recibirla en su constructor para pasársela al
hijo.',
            style: TextStyle(color: Colors.grey, fontSize: 13),
          ),
          const SizedBox(height: 20),

          // Prop Drilling: Pasa la función hacia el nivel 4
          TarjetaProductoNivel3(
            producto: productoEjemplo,
            onAgregarProducto: onAgregarProducto,
          ),
        ],
      ),
    );
  }
}

/// WIDGET NIVEL 4: Tarjeta de Producto (Hoja del árbol - Consumidor Final)
class TarjetaProductoNivel3 extends StatelessWidget {
```

```
  final Producto producto;
  final VoidCallback onAgregarProducto;

  const TarjetaProductoNivel3({
    super.key,
    required this.producto,
    required this.onAgregarProducto,
  });

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;

    return Card(
      elevation: 3,
      child: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Row(
          children: [
            Icon(Icons.book, size: 48, color: colorScheme.primary),
            const SizedBox(width: 16),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    producto.nombre,
                    style: const TextStyle(fontWeight: FontWeight.bold),
                  ),
                  Text('\$${producto.precio.toStringAsFixed(2)}'),
                ],
              ),
            ),

            // Dispara el callback que viaja 3 niveles hacia arriba
            ElevatedButton(
              onPressed: onAgregarProducto,
              child: const Text('Agregar'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Prácticas Guiadas / Laboratorio

Práctica 1: Refactorización e Identificación de Estado Efímero vs. Global

1. Analiza la pantalla del ejemplo fuente anterior.
2. Identifica cuáles variables pertenecen a la categoría de **estado efímero** (ej. la visibilidad de una tarjeta o la pestaña seleccionada) y cuáles representan un **estado global** (ej. el carrito de compras o la sesión del usuario).
3. Modifica la tarjeta del producto agregando una variable booleana local `_esFavorito` dentro de un `StatefulWidget` exclusivo para la tarjeta. Dispara `setState()` localmente para validar que conmuta su icono de corazón sin causar una reconstrucción del árbol superior.

Práctica 2: Análisis de Reconstrucciones de Widgets mediante `debugPrint`

1. Añade declaraciones de impresión en consola (`debugPrint('Reconstruyendo Widget X')`) en el método `build()` de cada uno de los niveles (`DemoPropDrillingApp`, `PantallaCatalogoNivel1`, `SeccionProductosNivel2`, `TarjetaProductoNivel3`).
2. Ejecuta la aplicación y presiona el botón "Agregar".
3. Observa en la consola como la llamada a `setState()` en la raíz fuerza la re-ejecución del método `build()` en **todos** los widgets intermedios, confirmando la ineficiencia del paradigma cuando la aplicación escala.

Resumen y Puntos Clave

- **Estado Efímero:** Datos locales contenidos dentro de un widget específico. Se administra adecuadamente utilizando `StatefulWidget` y `setState()`.
- **Estado Global:** Datos compartidos por múltiples componentes o pantallas en la aplicación. Requiere soluciones arquitectónicas de inyección de dependencias y reactividad.
- **Limitaciones de `setState()`:** Provoca acoplamiento entre la interfaz y la lógica de negocio, re-renderizados globales e ineficiencia en el consumo de recursos al cambiar el estado.
- **Prop Drilling:** Anti-patrón caracterizado por la transmisión manual de datos y *callbacks* a través de constructores en widgets intermedios que no utilizan dicha información, dificultando el mantenimiento del código.

Bibliografía y Referencias

1. **Google LLC.** (2024). *Ephemeral vs app state - Flutter Docs*. Recuperado de: https://docs.flutter.dev/data-and-backend/state-mgmt/ephemeral-vs-app
2. **Google LLC.** (2024). *Think declaratively - Flutter Docs*. Recuperado de: https://docs.flutter.dev/data-and-backend/state-mgmt/declarative
3. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.